

Index

Prog No	Problem statement
1	Creating “Hello world” Application
2	Creating an application that displays message based on screen orientation.
3	Create an application to develop Login windows using UI controls.
4	Create an application to implement new activities using explicit intent, implicit intent and content provider.
5	Create an application that displays custom designed Opening Screen.
6	Create an UI with all views.
7	Create menu in Application
8	Read/ write the Local data.
9	Create / Read / Write data with database (SQLite).
10	Create an application to send SMS and receive SMS
11	Create an application to send an e-mail.
12	Display Map based on the Current/given location.
13	Create a sample application with login module (check username and password) On successful login change Textview “Login Successful”. On login fail alert using Toast “login fail”
14	Learn to deploy Android applications

Step-by-step guide to create your first application in Android Studio

Step 1: Install Android Studio

1. Download **Android Studio** from the official site: <https://developer.android.com/studio>
 2. Install it by following the on-screen instructions.
-

Step 2: Start a New Android Project

1. Open **Android Studio**.
 2. Click on "**New Project**".
 3. Choose a project template: Select **Empty Activity** (simplest for beginners).
 4. Click **Next**.
-

Step 3: Configure the Project

1. **Name** your application (for example, "MyFirstApp").
 2. **Package Name** (usually in the format com.example.myfirstapp).
 3. Select **Save Location** on your system.
 4. Choose **Programming Language: Java** or **Kotlin**.
 5. Select **Minimum SDK** (for example, API 21 for Android 5.0 Lollipop).
 6. Click **Finish**.
-

Step 4: Understanding the Project Structure

- **MainActivity.java** (or **.kt**) → Main logic of the app.
 - **activity_main.xml** → UI layout of the app.
 - **AndroidManifest.xml** → Defines app permissions and activities.
 - **Gradle Files** → Manage dependencies and app settings.
-

Step 5: Design the User Interface (UI)

1. Open **activity_main.xml** under res/layout.
2. Switch to **Design Mode** or **Code Mode**.
3. Drag and drop a **Button** and **TextView** from the Palette.
4. Set the Button's text to "**Click Me**".
5. Set an **ID** for the Button (btnClick) and TextView (txtMessage).

Step 6: Add Functionality (Java/Kotlin Code)

1. Open **MainActivity.java** (app/java/com.example.myfirstapp).
2. Modify the code to handle button click (refer any sample code for the same).

Step 7: Run the App

1. Connect an **Android Device** (Enable **USB Debugging**) OR use the **Emulator**:
 - Go to **Tools > Device Manager**
 - Click **Create Virtual Device**, select a model, and install an Android System Image.
2. Click **Run** (▶ button) or press **Shift + F10**.

That's all!!

Program Number 1:

Creating “Hello world” Application

Java Code	MainActivity.java
<pre>package com.example.myapplication1; import android.os.Bundle; import androidx.appcompat.app.AppCompatActivity; public class MainActivity extends AppCompatActivity { @Override protected void onCreate(Bundle savedInstanceState) { super.onCreate(savedInstanceState); setContentView(R.layout.activity_main); } }</pre>	

Code explanation:

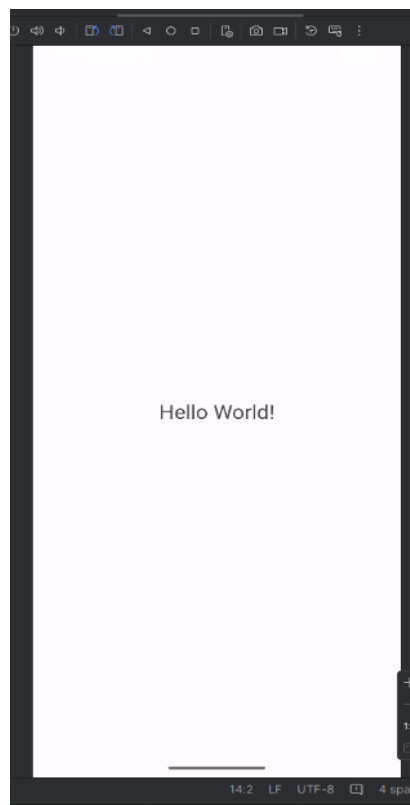
1. **Package Declaration:** Defines the package name for the app.
2. **Import Statements:** Import necessary classes for UI handling, button interaction, and displaying messages (Toast).
3. **Main Activity Class:** Defines MainActivity as the entry point of the application, extending AppCompatActivity for backward compatibility.
4. **onCreate Method:** Initializes the activity when it is launched.
5. **Enabling Edge-to-Edge Display (Optional):** Allows full-screen mode with edge-to-edge content display.
 - a. Displays a **Toast message** when the button is clicked.
6. **Summary:** This app displays a button, and when clicked, it shows a toast message saying, " **Hello World!**".

Code explanation:

1. **XML Declaration:** Specifies the version and encoding of the XML file.
2. **RelativeLayout:**
 - The root layout container that arranges child views in a linear direction (vertical or horizontal).
 - Uses xmlns attributes to define Android, app, and tools namespaces.
 - android:id="@+id/main" assigns a unique ID to the layout.
 - android:layout_width="match_parent" and android:layout_height="match_parent" make it occupy the full screen.
 - tools:context=".MainActivity" links this layout to MainActivity for previewing in the design editor.
3. Create a TextView

XML code	Activity_main.xml
<pre><?xml version="1.0" encoding="utf-8"?> <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android" xmlns:app="http://schemas.android.com/apk/res-auto" xmlns:tools="http://schemas.android.com/tools" android:id="@+id/main" android:layout_width="match_parent" android:layout_height="match_parent" tools:context=".MainActivity"> <TextView android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="Hello World!" android:textSize="24sp" android:layout_centerInParent="true"/> </RelativeLayout></pre>	

OUTPUT:



Program Number 2:

Creating an application that displays message based on screen orientation.

Java Code	MainActivity.java
<pre>package com.example.secondprogram; import android.os.Bundle; import android.view.WindowManager; import android.widget.TextView; import androidx.activity.EdgeToEdge; import androidx.appcompat.app.AppCompatActivity; import androidx.core.graphics.Insets; import androidx.core.view.ViewCompat; import androidx.core.view.WindowInsetsCompat; import android.graphics.Point; import android.view.Display; public class MainActivity extends AppCompatActivity { @Override protected void onCreate(Bundle savedInstanceState) { super.onCreate(savedInstanceState); setContentView(R.layout.activity_main); TextView orientationMessage = findViewById(R.id.orientation_message); //----get the current display info--- WindowManager wm = (WindowManager) getSystemService(WINDOW_SERVICE); Display d = wm.getDefaultDisplay(); Point size = new Point(); d.getSize(size); int width = size.x; int height = size.y; if (width > height) { //-----landscape Mode----- orientationMessage.setText("Landscape mode"); } else { //-----portrait mode----- orientationMessage.setText("Portrait mode"); } } }</pre>	

Code explanation:

1. **Package Declaration:** Defines the package name for the app.
2. **Import Statements:** Includes required classes for UI handling, screen orientation, button interaction, and displaying messages, widget.
3. **Main Activity Class:** MainActivity extends AppCompatActivity, making it the main entry point.

4. **onCreate Method:** Initializes the activity when launched.
5. **Setting the Layout:** Connects MainActivity to activity_main.xml
6. **Gets the screen size** using WindowManager.
7. **Retrieves width and height** of the screen.
 - **Compares them:**
 If **width > height**, it's **landscape mode** → Display "Landscape mode".
 Otherwise, it's **portrait mode** → Display "Portrait mode".

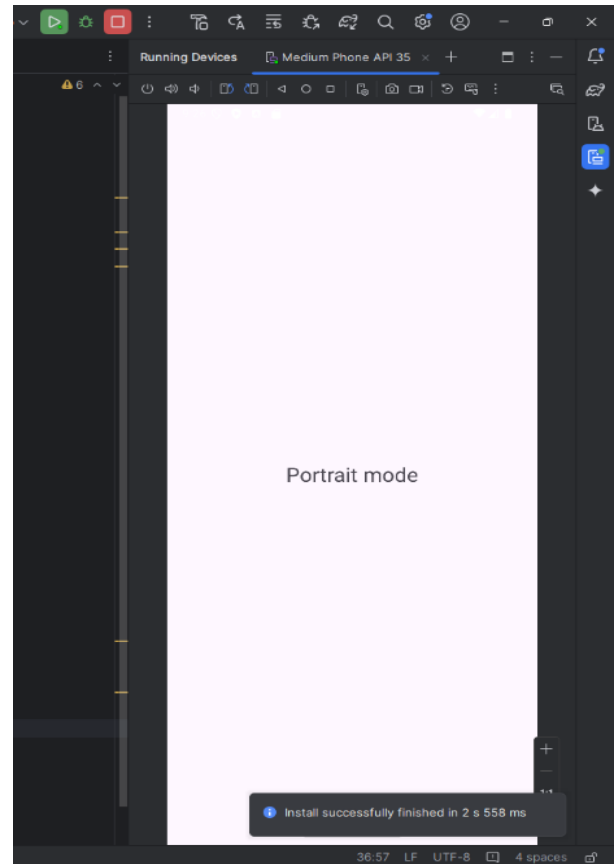
XML code	Activity_main.xml
<pre> <?xml version="1.0" encoding="utf-8"?> <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android" android:layout_width="match_parent" android:layout_height="match_parent" android:padding="16dp"> <TextView android:id="@+id/orientation_message" android:layout_width="wrap_content" android:layout_height="wrap_content" android:textSize="24sp" android:layout_centerInParent="true"/> </RelativeLayout> </pre>	

Code explanation:

This XML layout creates a **TextView** in the center of the screen to display the screen orientation ("Landscape mode" or "Portrait mode").

- **RelativeLayout:** Parent layout.
- **TextView** (id: orientation_message): Displays the orientation text.
- **layout_centerInParent="true"**: Centers the text in the middle.
- **textSize="24sp"**: Makes the text bigger

Output:



Program Number 3:

Create an application to develop Login windows using UI controls.

Java Code

```
package com.example.pg3;

import android.os.Bundle;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

Code explanation:

1. The XML layout defines a simple login screen with fields for username and password, plus a login button.
2. The `MainActivity` class sets this layout as the main view when the app starts.
3. It uses Android's `AppCompatActivity` to manage the UI lifecycle and interactions.

XML code

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <!-- Username Label -->
    <TextView
        android:id="@+id/label_username"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Username"
        android:textSize="18sp"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="50dp" />

    <!-- Username Input -->
    <EditText
        android:id="@+id/input_username"
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your username"
        android:layout_below="@id/label_username"
        android:layout_marginTop="10dp"
        android:layout_centerHorizontal="true" />

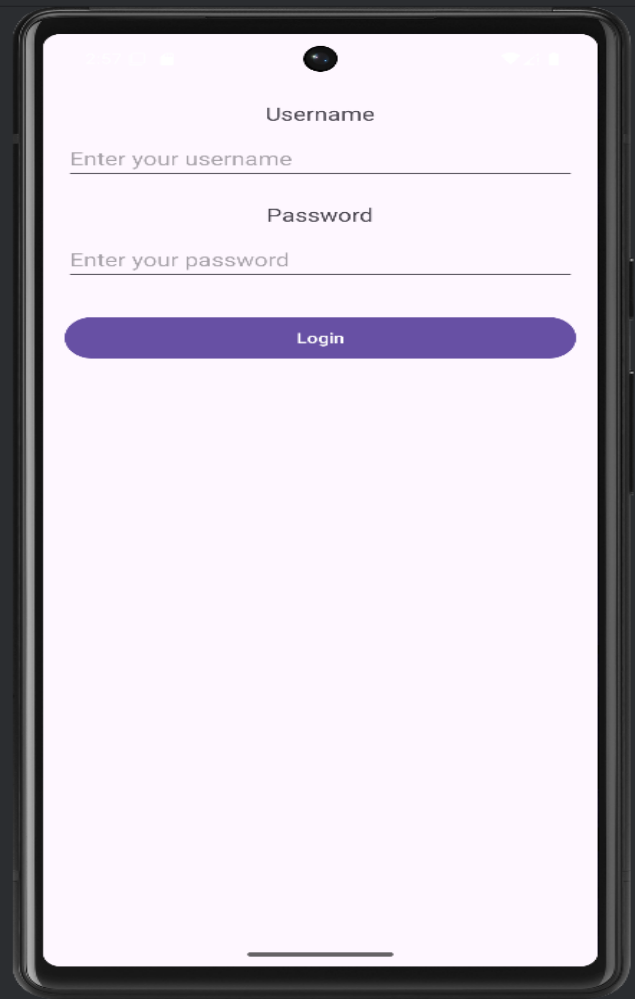
<!-- Password Label -->
<TextView
    android:id="@+id/label_password"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Password"
    android:textSize="18sp"
    android:layout_below="@id/input_username"
    android:layout_marginTop="20dp"
    android:layout_centerHorizontal="true" />

<!-- Password Input -->
<EditText
    android:id="@+id/input_password"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter your password"
    android:inputType="textPassword"
    android:layout_below="@id/label_password"
    android:layout_marginTop="10dp"
    android:layout_centerHorizontal="true" />

<!-- Login Button -->
<Button
    android:id="@+id/button_login"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Login"
    android:layout_below="@id/input_password"
    android:layout_marginTop="30dp"
    android:padding="10dp" />
</RelativeLayout>

```

Output



Username

Enter your username

Password

Enter your password

Login

Program Number 4:

Create an application to implement new activities using explicit intent, implicit intent and content provider.

1. Click New Project, the New Project Dialog box appears.
2. Choose Empty Views Activity then click Next.
3. Specify the Name of your project, Select the Language as Java, and Select the Minimum SDK as API 16 (“Jelly Bean”, Android 4.1). Click Finish Button.
4. To create another activity for Explicit Intent, Click File→New→Activity→ Empty Views Activity. A New Android Activity dialog box appears, Specify the Name of the activity as **NewActivity** then click Finish.

Code explanation:

- **onCreate()** – Sets up the app's screen (activity_main.xml).
- **onImplicitButtonClicked()** – Opens **Google** in a web browser using an **implicit intent**.
- **onExplicitButtonClicked()** – Switches to NewActivity using an **explicit intent**.

Java Code –	MainActivity.java
<pre>package com.example.fourthprogram; import android.content.Intent; import android.net.Uri; import android.os.Bundle; import android.view.View; import androidx.activity.EdgeToEdge; import androidx.appcompat.app.AppCompatActivity; import androidx.core.graphics.Insets; import androidx.core.view.ViewCompat; import androidx.core.view.WindowInsetsCompat; public class MainActivity extends AppCompatActivity { @Override protected void onCreate(Bundle savedInstanceState) { super.onCreate(savedInstanceState); EdgeToEdge.enable(this); setContentView(R.layout.activity_main); } public void onImplicitButtonClicked(View view) { Uri url=Uri.parse("https://www.google.com"); Intent i=new Intent(Intent.ACTION_VIEW, url); startActivity(i); } public void onExplicitButtonClicked(View view) { Intent i=new Intent(MainActivity.this, NewActivity.class); startActivity(i); } }</pre>	

XML code – activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    tools:context=".MainActivity">

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Implicit Intent"
        android:onClick="onImplicitButtonClicked"
    />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Explicit Intent"
        android:onClick="onExplicitButtonClicked"/>

</LinearLayout>
```

Code explanation:

1. **XML Declaration** – The file starts with an XML declaration specifying version 1.0 and UTF- 8 encoding.
2. **Root Layout (LinearLayout)** – Defines a vertical layout that stacks elements from top to bottom.
3. **Namespace Declarations** – Includes standard Android, auto-generated, and tools namespaces for attributes.
4. **Layout Properties** – The LinearLayout takes up the full width and height of the parent container.
5. **Context Reference** – The tools:context attribute links this layout to MainActivity for preview purposes in the design editor.
6. **Button Element** – A button is placed inside the LinearLayout.
7. **First Button ("Implicit Intent")**
 - When clicked, it calls onImplicitButtonClicked(), which **opens Google in a browser**.
8. **Second Button ("Explicit Intent")**
 - When clicked, it calls onExplicitButtonClicked(), which **opens another activity (NewActivity)**.
 - ◆ android:onClick="methodName" links the button to its function in Java/Kotlin.
 - ◆ wrap_content adjusts the button size to fit the text.

NOTE:

To create another activity for Explicit Intent, Click File→New→Activity→ Empty Views Activity. A New Android Activity dialog box appears, Specify the Name of the activity as NewActivity then click Finish.

Code explanation:

For the new “Activity.Java page”

NOTE: ***Don't add anything to activity_new.java or Don't Remove the page leave the page as it is

XML code – activity_new.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".NewActivity">

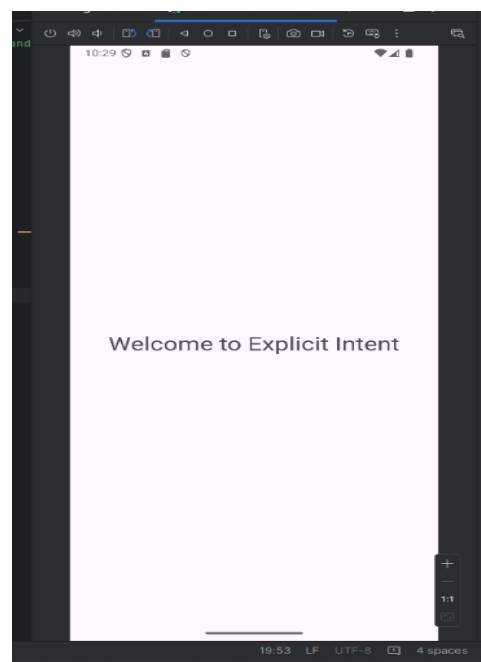
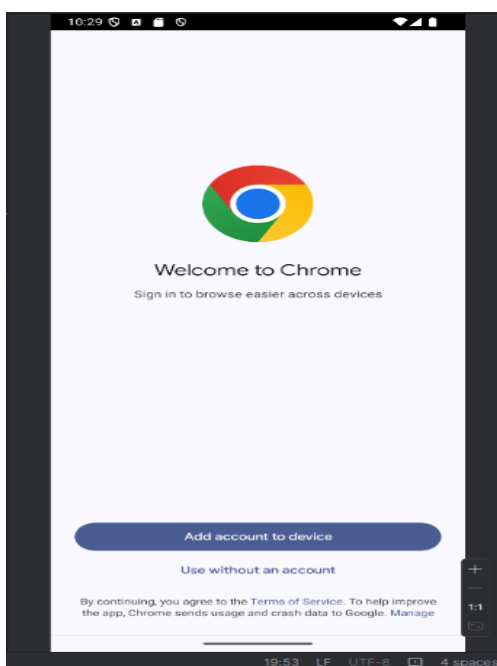
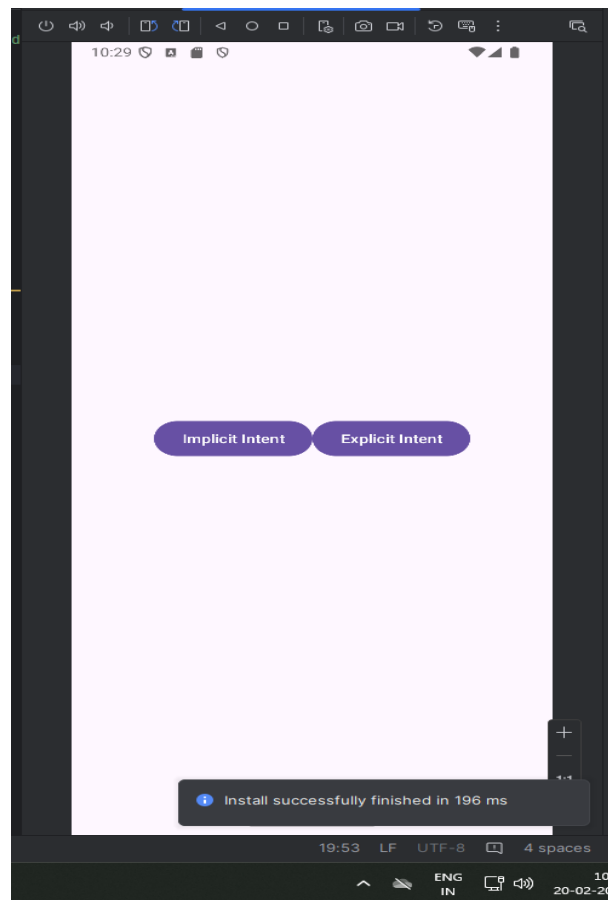
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome to Explicit Intent"
        android:textSize="28sp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Code explanation:

1. **XML Declaration** – The file starts with an XML declaration specifying version 1.0 and UTF- 8 encoding.
2. **Root Layout** – A LinearLayout is used as the root container, which arranges child views in a linear fashion (either vertically or horizontally).
3. **Namespace Declarations** – It includes standard Android, auto, and tools namespaces to define attributes and provide development-time hints.
4. **Layout Properties** – The ConstraintLayout takes up the complex layout with flat view hierarchy.
5. text="Welcome to Explicit Intent" → Sets the displayed text.
6. textSize="28sp" → Makes the text large.
7. wrap_content → Adjusts width and height based on text size.

8. Constraints (app:layout_constraint...) → Centers the text in the parent layout.



Program Number 5:

Create an application that displays custom designed Opening Screen.

Java Code

```
package com.example.lab5;

import android.os.Bundle;
import android.os.Handler;
import android.view.View;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private static final int SPLASH_DISPLAY_LENGTH = 10000; // 10 seconds

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        final View splashScreen = findViewById(R.id.splash_screen);
        final View mainContent = findViewById(R.id.main_content);

        new Handler().postDelayed(new Runnable() {
            @Override
            public void run() {
                splashScreen.setVisibility(View.GONE);
                mainContent.setVisibility(View.VISIBLE);
            }
        }, SPLASH_DISPLAY_LENGTH);
    }
}
```

Activity and Setup

- MainActivity extends AppCompatActivity, making it the main screen of the app.
- The layout used is activity_main.xml.

Splash Screen Duration

- A constant SPLASH_DISPLAY_LENGTH is set to 10000 milliseconds → **10 seconds**.
1. **Set the UI Layout:**
 - setContentView(R.layout.activity_main) loads the screen design from XML.

2. Find Views:

- splashScreen → View with ID splash_screen.
- mainContent → View with ID main_content.

Handler to Delay the Transition

3. A Handler is used with postDelayed() to schedule a task after 10 seconds.
 4. Inside the run() method of the Runnable:
 - splashScreen.setVisibility(View.GONE) → Hides the splash screen.
 - mainContent.setVisibility(View.VISIBLE) → Shows the main content
- To create a **splash screen effect**:
 - Show splash for 10 seconds.
 - Then reveal the main app screen

XML code

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

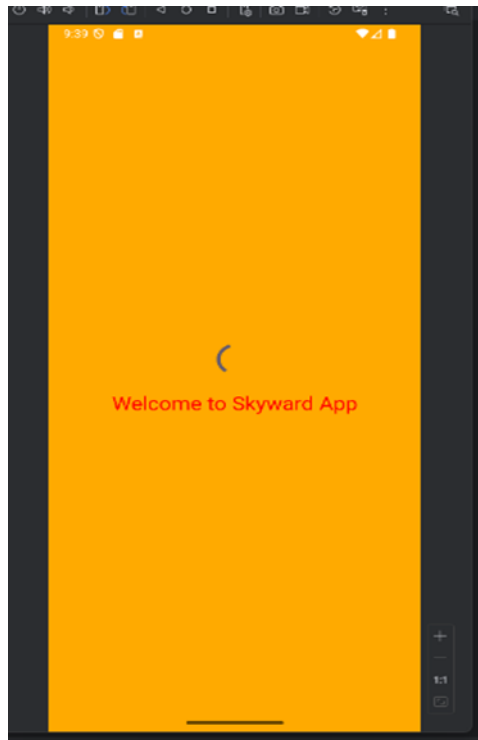
    <!-- Splash Screen Layout -->
    <RelativeLayout
        android:id="@+id/splash_screen"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:background="#FFAA00"
        android:padding="16dp"
        android:gravity="center">

        <ProgressBar
            android:id="@+id/progress_bar"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_centerHorizontal="true"
            android:indeterminate="true" />

        <TextView
            android:id="@+id/app_name"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Welcome to Skyward App"
            android:textSize="24sp"
            android:textColor="#FF0000"
            android:layout_below="@id/progress_bar"
            android:layout_centerHorizontal="true"
            android:layout_marginTop="16dp" />
    </RelativeLayout>

    <!-- Main Content Layout -->
    <LinearLayout
        android:id="@+id/main_content"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:padding="16dp"
        android:gravity="center"
        android:visibility="gone">

        <TextView
            android:id="@+id/welcome_text"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Welcome to the Main Activity Screen"
            android:textSize="18sp"
            android:textColor="#000000" />
    </LinearLayout>
</FrameLayout>
```



Program Number 6:

Create an UI with all views.

Java Code

```
package com.example.pg6;

import android.os.Bundle;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.DatePicker;
import android.widget.EditText;
import android.widget.NumberPicker;
import android.widget.ProgressBar;
import android.widget.RadioButton;
import android.widget.RadioGroup;
import android.widget.Spinner;
import android.widget.Switch;
import android.widget.TextView;
import android.widget.TimePicker;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private TextView textView;
    private EditText editText;
    private Button button;
    private CheckBox checkBox;
    private RadioGroup radioGroup;
    private RadioButton radioButton1, radioButton2;
    private Switch switch1;
    private Spinner spinner;
    private ProgressBar progressBar;
    private DatePicker datePicker;
    private TimePicker timePicker;
    private NumberPicker numberPicker;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Link UI components
        textView = findViewById(R.id.textView1);
        editText = findViewById(R.id.editText1);
        button = findViewById(R.id.button1);
        checkBox = findViewById(R.id.checkBox1);
        radioGroup = findViewById(R.id.radioGroup1);
        radioButton1 = findViewById(R.id.radioButton2);
        radioButton2 = findViewById(R.id.radioButton3);
        switch1 = findViewById(R.id.switch11);
        spinner = findViewById(R.id.spinner12);
    }
}
```

```
        progressBar = findViewById(R.id.progressBar13);
        datePicker = findViewById(R.id.datePicker14);
        timePicker = findViewById(R.id.timePicker15);
        numberPicker = findViewById(R.id.numberPicker16);

        // Additional configuration for NumberPicker
        numberPicker.setMinValue(0);
        numberPicker.setMaxValue(100);
    }
}
```

Code explanation:

- This Java code defines the MainActivity class for an Android app, extending AppCompatActivity.
- It connects Java objects to their corresponding UI elements from the XML layout using findViewById().
- The layout includes widgets like TextView, EditText, Button, CheckBox, RadioGroup, Switch, Spinner, and more.
- It initializes and configures a NumberPicker to allow values from 0 to 100.
- The code sets up the UI but doesn't yet include event handling or user interaction logic.

XML code

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:orientation="vertical"
        android:padding="16dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <TextView
            android:id="@+id/textView1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="This is a TextView"
            android:textSize="18sp"
            android:layout_marginBottom="16dp" />

        <EditText
            android:id="@+id/editText1"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Enter your text here"
            android:layout_marginBottom="16dp" />

        <Button
            android:id="@+id/button1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Click Me"
            android:layout_marginBottom="16dp" />

        <CheckBox
            android:id="@+id/checkbox1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Check Me"
            android:layout_marginBottom="16dp" />

        <RadioGroup
            android:id="@+id/radioGroup1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:layout_marginBottom="16dp">

            <RadioButton
                android:id="@+id/radioButton2"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Option 1" />
```

```

        <RadioButton
            android:id="@+id/radioButton3"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Option 2" />
    </RadioGroup>

    <Switch
        android:id="@+id/switch11"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Switch"
        android:layout_marginBottom="16dp" />

    <Spinner
        android:id="@+id/spinner12"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="16dp" />

    <ProgressBar
        android:id="@+id/progressBar13"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="16dp" />

    <DatePicker
        android:id="@+id/datePicker14"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="16dp" />

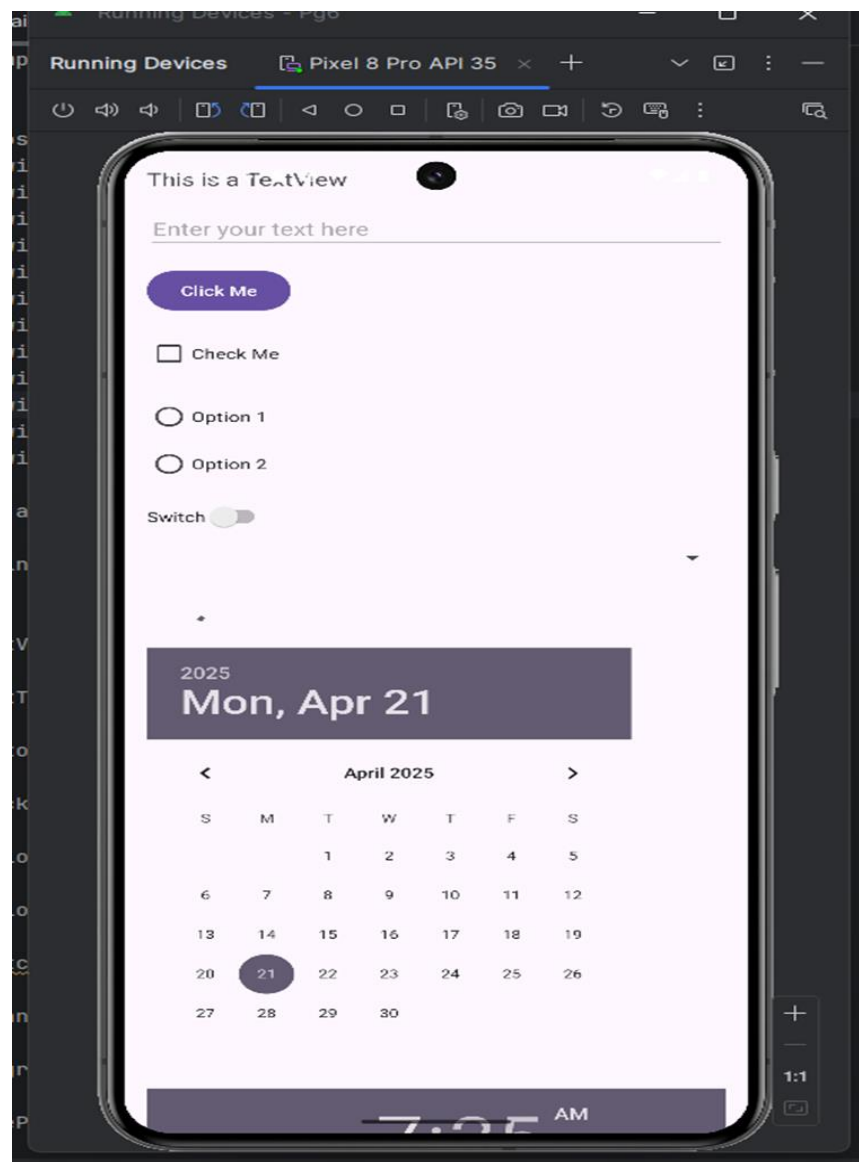
    <TimePicker
        android:id="@+id/timePicker15"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="16dp" />

    <NumberPicker
        android:id="@+id/numberPicker16"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="16dp" />
    </LinearLayout>
</ScrollView>

```

Code explanation:

- This XML layout creates a **scrollable user interface** using a `ScrollView` containing a `LinearLayout` with vertical orientation.
- Inside, it includes common UI elements like a `TextView`, `EditText`, `Button`, and `CheckBox` for displaying and taking input.
- A `RadioGroup` holds two `RadioButton` options for single-choice selection.
- Additional widgets like `Switch`, `Spinner`, `ProgressBar`, `DatePicker`, `TimePicker`, and `NumberPicker` are also included for various inputs.
- The layout is designed to be vertically scrollable, ensuring all elements are accessible on smaller screens.



Program Number 7:

Create menu in Application

Steps includes

1. Click New Project, the New Project Dialog box appears.
2. Choose Empty Views Activity then click Next.
3. Specify the Name of your project, Select the Language as Java, and Select the Minimum SDK as API 16 ("Jelly Bean", Android 4.1). Click Finish Button.

5. To create a Menu Resource File:

Right-click on the res directory in your Android project, navigate to **New > Android Resource File**, and name the file **menus.xml**, Root element as **Menu** and update the following content.

Menus.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
<item android:id="@+id/php"
android:title="PHP"/>
<item android:id="@+id/java"
android:title="JAVA"/>
<item android:id="@+id/csharp"
android:title="C#"/>
</menu>
```

Code Explanation:

- This XML file defines a menu resource for an Android app.
- It uses the <menu> root tag and contains three <item> elements.
- Each <item> represents a clickable menu option in the app's UI.
- The android:id uniquely identifies each menu item (php, java, csharp).
- The android:title specifies the label that will be displayed for each item in the menu.

Java Code

```
package com.example.prg7;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuInflater;
import android.view.MenuItem;
import android.widget.Toast;
import androidx.activity.EdgeToEdge;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends
AppCompatActivity {

    @Override

    public boolean onCreatePanelMenu(int featureId,
@NonNull Menu menu) {

        MenuInflater inflater=getMenuInflater();

        inflater.inflate(R.menu.menus,menu);

        return true;

    }

    @Override

    public boolean onOptionsItemSelected(@NonNull
MenuItem item) {

        if(item.getItemId()==R.id.php) {

            Toast.makeText(this, "Php Page",
Toast.LENGTH_SHORT).show();

        }

        if(item.getItemId()==R.id.java) {

            Toast.makeText(this, "Java Page",
```

```

Toast.LENGTH_SHORT).show();

    }

    if(item.getItemId()==R.id.csharp) {

        Toast.makeText(this, "C# Page",
Toast.LENGTH_SHORT).show();

    }

    return super.onOptionsItemSelected(item);

}

}

```

Code explanation:

1. This Android MainActivity sets up an options menu using a custom XML (menus.xml) and shows a toast message based on the selected menu item (PHP, Java, or C#).
2. The onCreatePanelMenu() inflates the menu when the panel is created.
3. The onOptionsItemSelected() handles user clicks on the menu items and displays a corresponding toast message.

XML code : AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.INTERNET" />
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.AppCompat.Light"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>

```



Program Number 8:

Read / Write data the Local Data

Java Code

```
package com.example.pg9;

import android.content.Context;
import android.content.SharedPreferences;
import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.Spinner;
import android.widget.Switch;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private Spinner languageSpinner;
    private Switch notificationsSwitch;
    private TextView preferencesTextView;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        languageSpinner = findViewById(R.id.languageSpinner);
        notificationsSwitch = findViewById(R.id.notificationsSwitch);
        preferencesTextView = findViewById(R.id.preferencesTextView);
        Button saveButton = findViewById(R.id.saveButton);

        ArrayAdapter<CharSequence> adapter =
        ArrayAdapter.createFromResource(
            this, R.array.languages,
            android.R.layout.simple_spinner_item);

        adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
        languageSpinner.setAdapter(adapter);

        loadUserPreferences();

        saveButton.setOnClickListener(v -> saveUserPreferences());
    }

    private void saveUserPreferences() {
        String language = languageSpinner.getSelectedItem().toString();
        boolean notificationsEnabled = notificationsSwitch.isChecked();

        SharedPreferences sharedPreferences =
        getSharedPreferences("UserPreferences", Context.MODE_PRIVATE);
        SharedPreferences.Editor editor = sharedPreferences.edit();
```

```

        editor.putString("language", language);
        editor.putBoolean("notificationsEnabled", notificationsEnabled);
        editor.apply();

        preferencesTextView.setText(String.format("Language:
%s\nNotifications Enabled: %b",
            language, notificationsEnabled));
    }

    private void loadUserPreferences() {
        SharedPreferences sharedPreferences =
getSharedPreferences("UserPreferences", Context.MODE_PRIVATE);
        String language = sharedPreferences.getString("language",
"English");
        boolean notificationsEnabled =
sharedPreferences.getBoolean("notificationsEnabled", false);

        ArrayAdapter<CharSequence> adapter = (ArrayAdapter<CharSequence>)
languageSpinner.getAdapter();
        int spinnerPosition = adapter.getPosition(language);
        languageSpinner.setSelection(spinnerPosition);

        notificationsSwitch.setChecked(notificationsEnabled);

        preferencesTextView.setText(String.format("Language:
%s\nNotifications Enabled: %b",
            language, notificationsEnabled));
    }
}

```

Code explanation:

1. This code defines an Android activity that allows users to select a language and toggle notifications using a spinner and a switch.
2. User preferences are saved using `SharedPreferences` when the "Save" button is clicked.
3. The spinner is populated with language options from a string array resource (`R.array.languages`).
4. When the app starts, saved preferences (if any) are loaded and displayed in a `TextView`.
5. Changes made by the user are shown and persisted for future sessions using `saveUserPreferences()`.
 - Appends file content to a `StringBuffer`.
 - Displays the content in a toast message.
6. **Exception Handling:** Both file operations handle exceptions, ensuring the app does not crash due to file access issues.

**** To get the string.xml go to res-> value -> string.xml ,
and paste the highlighted code**

String.xml

```
<resources>
    <string name="app_name">pg9</string>
    <string-array name="languages">
        <item>English</item>
        <item>Spanish</item>
        <item>French</item>
        <item>German</item>
        <item>Chinese</item>
    </string-array>
</resources>
```

XML code

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/languageLabel"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Select Language" />

    <Spinner
        android:id="@+id/languageSpinner"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:entries="@array/languages" />

    <TextView
        android:id="@+id/notificationsLabel"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Enable Notifications" />

    <Switch
        android:id="@+id/notificationsSwitch"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

    <Button
```

```
        android:id="@+id/saveButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Save Preferences" />

        <TextView
            android:id="@+id/preferencesTextView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:paddingTop="16dp"
            android:text="Preferences will be displayed here" />

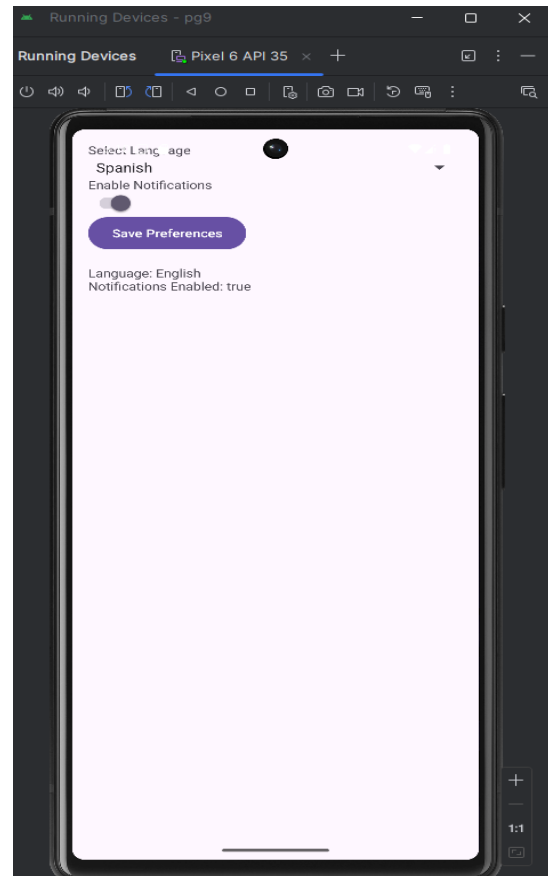
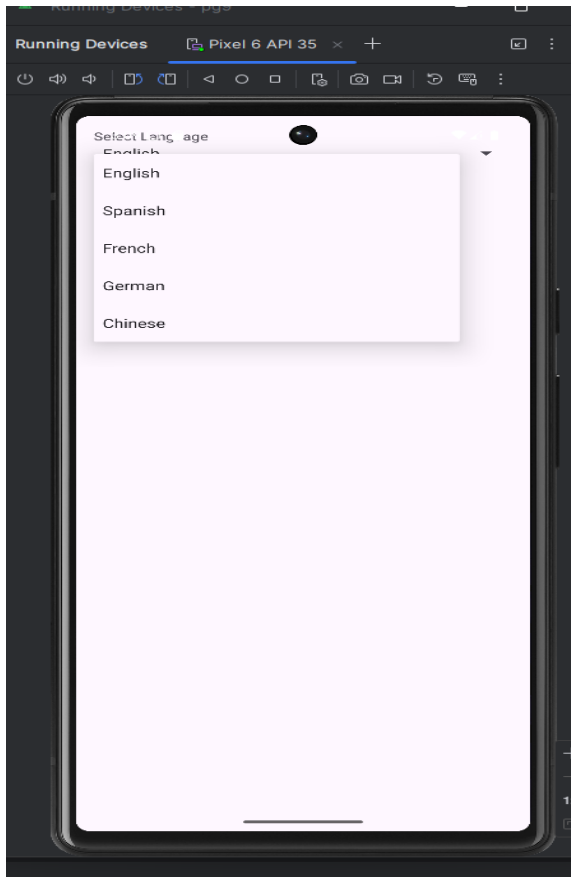
    </LinearLayout>
```

Code explanation:

Here's a simple explanation of your XML layout:

1. **Root Layout:** A `LinearLayout` with vertical orientation arranges child views from top to bottom, with padding around the edges.
2. **Language Selection:** A `TextView` displays the label "Select Language", and a `Spinner` allows users to choose a language from the `@array/languages` resource.
3. **Notification Toggle:** Another `TextView` labeled "Enable Notifications" is followed by a `Switch` that users can toggle on or off.
4. **Save Button:** A `Button` labeled "Save Preferences" triggers saving the selected language and switch state when clicked.
5. **Display Preferences:** A final `TextView` displays the current preferences (language and notification status) after saving or loading.

This layout works with your `MainActivity.java` to store and show user preferences.



Program Number 9:

Create / Read / Write data with database (SQLite).

XML code

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Student Details"
        android:textSize="25dp"
        android:layout_centerHorizontal="true"/>

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Rollno"
        android:id="@+id/et1"
        android:layout_marginTop="50dp"/>

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Name"
        android:id="@+id/et2"
        android:layout_marginTop="35dp"
        android:layout_below="@id/et1"
        android:layout_centerHorizontal="true"/>

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Department"
        android:id="@+id/et3"
        android:layout_marginTop="35dp"
        android:layout_below="@id/et2"
        android:layout_centerHorizontal="true"/>

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/b1"
        android:layout_below="@id/et3"
        android:text="Update"
        android:onClick="onUpdate"
        android:textSize="20dp"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="30dp"/>

    <Button
```

```

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/b2"
        android:layout_below="@id/b1"
        android:text="Read"
        android:onClick="onRead"
        android:textSize="20dp"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="30dp"/>
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/b3"
        android:layout_below="@id/b2"
        android:text="Insert"
        android:onClick="onInsert"
        android:textSize="20dp"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="30dp"/>
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/b4"
        android:layout_below="@id/b3"
        android:text="Delete"
        android:onClick="onDelete"
        android:textSize="20dp"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="30dp"/>

```

```
</RelativeLayout>
```

Java Code(MainActivity.java)

```

package com.example.sql1;

import android.content.ContentValues;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {
    SQLiteDatabase db;

```

```

EditText et1,et2,et3;
Button b1,b2,b3,b4;
String rno;
String name;
String dept;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    et1=findViewById(R.id.et1);
    et2=findViewById(R.id.et2);
    et3=findViewById(R.id.et3);
    b1=findViewById(R.id.b1);
    b2=findViewById(R.id.b2);
    b3=findViewById(R.id.b3);
    b4=findViewById(R.id.b4);
    DBHelper dbHelper=new DBHelper(this);
    db=dbHelper.getReadableDatabase();
    db=dbHelper.getWritableDatabase();

    }
    public void onUpdate(View view) {
        rno=et1.getText().toString();
        name=et2.getText().toString();
        dept=et3.getText().toString();
        if(rno.equals("") || name.equals("") || dept.equals(""))
        {
            Toast.makeText(this, "Please enter values",
Toast.LENGTH_LONG).show();
            return;
        }else {
            ContentValues values = new ContentValues();
            values.put("rollno", rno);
            values.put("name", name);
            values.put("dept", dept);
            db.update("student", values,
                "rollno="+rno,null);
            Toast.makeText(this, "Values Inserted" +
                "Successfully", Toast.LENGTH_LONG).show();

        } }

    public void onRead(View view) {
        StringBuffer buffer=new StringBuffer();
        Cursor c=db.rawQuery("select * from student",
            null);
        while(c.moveToNext()) {
            buffer.append("\n" + c.getString(0));
            buffer.append("\n" + c.getString(1));
            buffer.append("\n" + c.getString(2));
        }
        Toast.makeText(this,buffer.toString(),
Toast.LENGTH_LONG).show();
    }

```

```

    }

    public void onInsert(View view) {
        rno=et1.getText().toString();
        name=et2.getText().toString();
        dept=et3.getText().toString();
        if(rno.equals("") || name.equals("") || dept.equals(""))
        {
            Toast.makeText(this, "Please enter values",
Toast.LENGTH_LONG).show();
            return;
        }else {
            ContentValues values = new ContentValues();
            values.put("rollno", rno); values.put("name",
            name); values.put("dept", dept);
            db.insert("student", null, values);
            Toast.makeText(this, "Values Inserted" +
                "Successfully", Toast.LENGTH_LONG).show();

        }
    }

    public void onDelete(View view) {
        rno=et1.getText().toString();
        name=et2.getText().toString();
        dept=et3.getText().toString();
        if(rno.equals(""))
        {
            Toast.makeText(this,"Please enter roll no",
                Toast.LENGTH_LONG).show();
            return;} else{
            db.delete("student","rollno="+rno,
                null);
            Toast.makeText(this,"Values Deleted Successfully",
                Toast.LENGTH_LONG).show();

        }
    }
}

```

1. Go to main→java→com.example.program10,Right click →new→Java class→DBhelper, select class.

Java Code(DBhelper.java)

```

package com.example.sql1;

import android.content.Context;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

```

```

import androidx.annotation.Nullable;

public class DBHelper extends SQLiteOpenHelper {

    public DBHelper(@Nullable Context context) { super(context,
        "student", null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase sqLiteDatabase) {
        sqLiteDatabase.execSQL("create table " +
            "student(rollno int,name varchar(20)," + "dept
            varchar(5))");
    }

    @Override
    public void onUpgrade(SQLiteDatabase sqLiteDatabase, int i, int i1) {
        sqLiteDatabase.execSQL("drop table if exists student");
        onCreate(sqLiteDatabase);
    }
}

```

Code explanation:

1. Activity_main.xml gives the UI design .
2. Main_activity.java gives the code for creating an instance
3. DBHelper.java is required to connect to sqlite.

Program Number 10:

Create an application to Send and Receive SMS

Activity_XML code

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Enter the Phone Number"
        android:layout_marginTop="30dp"
        android:textSize="20dp"
        android:layout_marginStart="200dp"/>
    <EditText
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:id="@+id/ed1"
        android:phoneNumber="true" />
    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:id="@+id/tv1"
        android:text="Enter Message"
        android:textSize="30dp"
        android:layout_marginTop="50dp" />
    <EditText
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:id="@+id/ed2"
        android:inputType="textMultiLine"
        android:lines="5"
        android:gravity="center"/>
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/send"
        android:gravity="center"
        android:textSize="30dp"
        android:text="SEND"/>
</LinearLayout>
```

Code Explanation:

1. **TextView** → Displays "Enter the Phone Number".
2. **EditText (ed1)** → Allows the user to enter a phone number.
3. **TextView** → Displays "Enter Message".
4. **EditText (ed2)** → Lets the user type a message (multi-line).
5. **Button (send)** → When clicked, it will send the message.

◆ LinearLayout arranges everything **vertically**.

◆ fill_parent/match_parent makes elements **stretch** across the screen.

Android Manifest File needs permissions to be set

AndroidManifest.XML code

```
<?xml version="1.0" encoding="utf-8"?>

<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-feature android:name="android.hardware.telephony"
        android:required="false"/>

    <uses-permission android:name="android.permission.RECEIVE_SMS"/>
    <uses-permission android:name="android.permission.SEND_SMS"/>
    <uses-permission android:name="android.permission.READ_SMS"/>
    <uses-permission android:name="android.permission.READ_PHONE_STATE"/>
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
```



```
    android:theme="@style/Theme.SMS"

    tools:targetApi="31">

    <activity

        android:name=".MainActivity"

        android:exported="true">

        <intent-filter>

            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />

        </intent-filter>

    </activity>

    <receiver android:name=".smsReceiver"

        android:exported="true"

        android:permission="android.permission.BROADCAST_SMS">

        <intent-filter>

            <action android:name="android.provider.Telephony.SMS_RECEIVED"/>

        </intent-filter>

    </receiver>

</application>

</manifest>
```

Code Explanation:

- **Permissions** (Allows the app to handle SMS & calls)
 - RECEIVE_SMS → Receive incoming SMS.
 - SEND_SMS → Send SMS messages.
 - READ_SMS → Read SMS messages.
 - READ_PHONE_STATE → Access phone status.
 - INTERNET → Use the internet.
- **MainActivity** (App Entry Point)
 - Defined as the **main launcher activity**.
- **SMS Receiver (smsReceiver)**
 - Listens for incoming SMS (SMS_RECEIVED intent).
 - Needs **BROADCAST_SMS** permission to receive SMS.

Main_activity.java contains the code to send SMS

Activity_main.java

```
package com.example.sms;

import static android.app.ProgressDialog.show;
import android.app.PendingIntent;
import android.content.Intent;
import android.os.Bundle;
import android.telephony.SmsManager;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
```

```

import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        EdgeToEdge.enable(this);

        setContentView(R.layout.activity_main);

        EditText phone=findViewById(R.id.ed1);

        EditText msg=findViewById(R.id.ed2);

        Button send=findViewById(R.id.send);

        send.setOnClickListener(new View.OnClickListener() {

            @Override

            public void onClick(View view) {

                String phoneno=phone.getText().toString();

                String message=msg.getText().toString();

                SmsManager SmsManager=
android.telephony.SmsManager.getDefault();

                Intent intent=new
Intent(getApplicationContext(),MainActivity.class);

                PendingIntent
pi=PendingIntent.getActivity(getApplicationContext(), 0,intent,
PendingIntent.FLAG_IMMUTABLE);

                SmsManager.sendTextMessage(phoneno,null,message,pi,null);

                Toast.makeText(getApplicationContext(),"message sent
succefully",Toast.LENGTH_LONG).show();

            }

        });

    }

}

```

Code Explanation:

1. UI Setup:

- Takes a **phone number** (ed1) and a **message** (ed2) as input.
- A **Send button** (send) is linked to an action.

2. Button Click Action:

- Gets the **phone number** and **message** from input fields.
- Uses **SmsManager** to send an SMS.
- Creates a **PendingIntent** to handle message status.
- Displays a **Toast message** confirming SMS sent.

Uses SmsManager → Sends an SMS programmatically.

PendingIntent → Handles message delivery.

Toast Notification → Confirms message was sent successfully.

Go to Main □ **java** □ **com.example.program10**. **Right click** □ **New** □ **Java Class** and create new java file **smsReceiver**.

Code Explanation

1. **Triggered when an SMS is received.**
2. **Extracts SMS data** from the received Intent.
3. **Retrieves sender's phone number** and message content.
4. **Displays the message** as a Toast notification.

Key Features:

BroadcastReceiver → Listens for SMS messages.

Extracts SMS using pdu (Protocol Data Units).

Displays message using Toast.

Ensure while executing, SMS feature is allowed for the app.

smsReceiver.java

```
package com.example.sms;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.os.Bundle;
import android.telephony.SmsMessage;
import android.widget.Toast;

public class smsReceiver extends BroadcastReceiver {

    @Override
    public void onReceive(Context context, Intent intent) {

        Bundle bundle=intent.getExtras();

        SmsMessage[] msg=null;

        String str="SMS From";

        if(bundle!=null)
        {

            Object[] recv=(Object[]) bundle.get("pdus");

            msg=new SmsMessage[recv.length];

            for (int i=0;i<msg.length;i++) {

                msg[i] = SmsMessage.createFromPdu((byte[]) recv[i]);

                if (i==0) {

                    str += msg[i].getOriginatingAddress();

                    str += " ";

                }

                str += msg[i].getMessageBody().toString();

            }

        }

    }

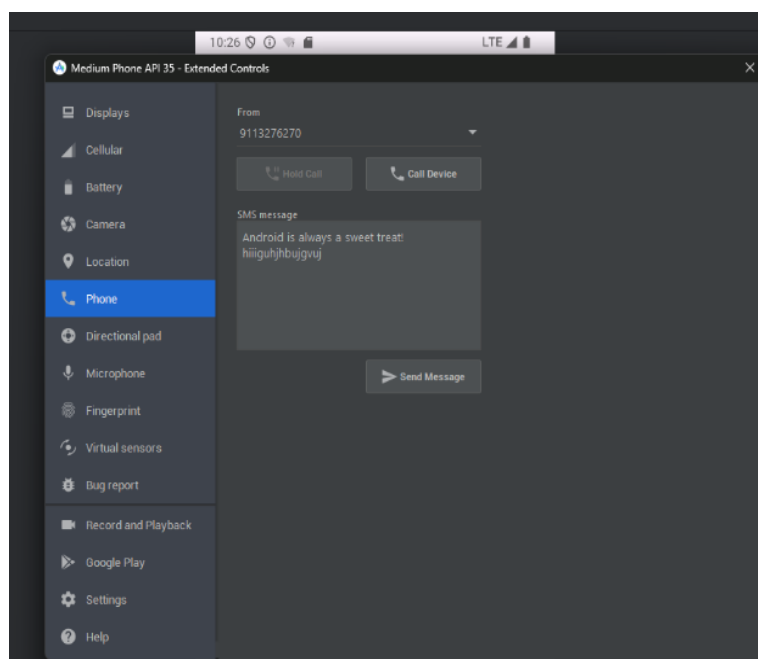
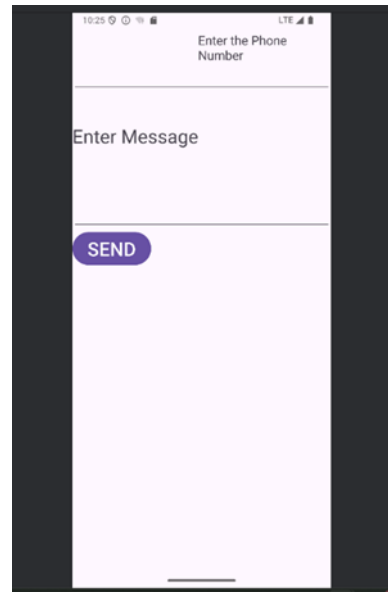
}
```

```
Toast.makeText(context,str,Toast.LENGTH_LONG).show();
```

```
}
```

```
}
```

```
}
```



Program Number 11:

Create an application to Send an email

XML code

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Subject"
        android:id="@+id/sub"
        android:layout_marginTop="50dp"/>
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Content"
        android:id="@+id/con"
        android:layout_below="@+id/sub"
        android:layout_marginTop="50dp"/>
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter MailAddress"
        android:id="@+id/mail"
        android:layout_below="@+id/con"
        android:layout_marginTop="50dp"/>
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/send"
        android:text="Send Email!"
```

```
android:layout_below="@id/mail"
android:layout_centerHorizontal="true"/>
```

```
</RelativeLayout>
```

Main_activity.java

```
package com.example.bca_program11;

import static android.content.Intent.createChooser;

import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {
    Button b1;
    EditText ed1,ed2,ed3;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        ed1=findViewById(R.id.sub);
        ed2=findViewById(R.id.con);
        ed3=findViewById(R.id.mail);
        b1=findViewById(R.id.send);
        b1.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String subject,content,to_email;
                subject=ed1.getText().toString();
                content=ed2.getText().toString();
                to_email=ed3.getText().toString();
                if(subject.equals("") && content.equals("") &&
to_email.equals(""))
                {
                    Toast.makeText(MainActivity.this,"All Fields are
required",Toast.LENGTH_LONG).show();
                }
                else{
                    sendmail(subject,content,to_email);
                }
            }
        })
    }
}
```



```

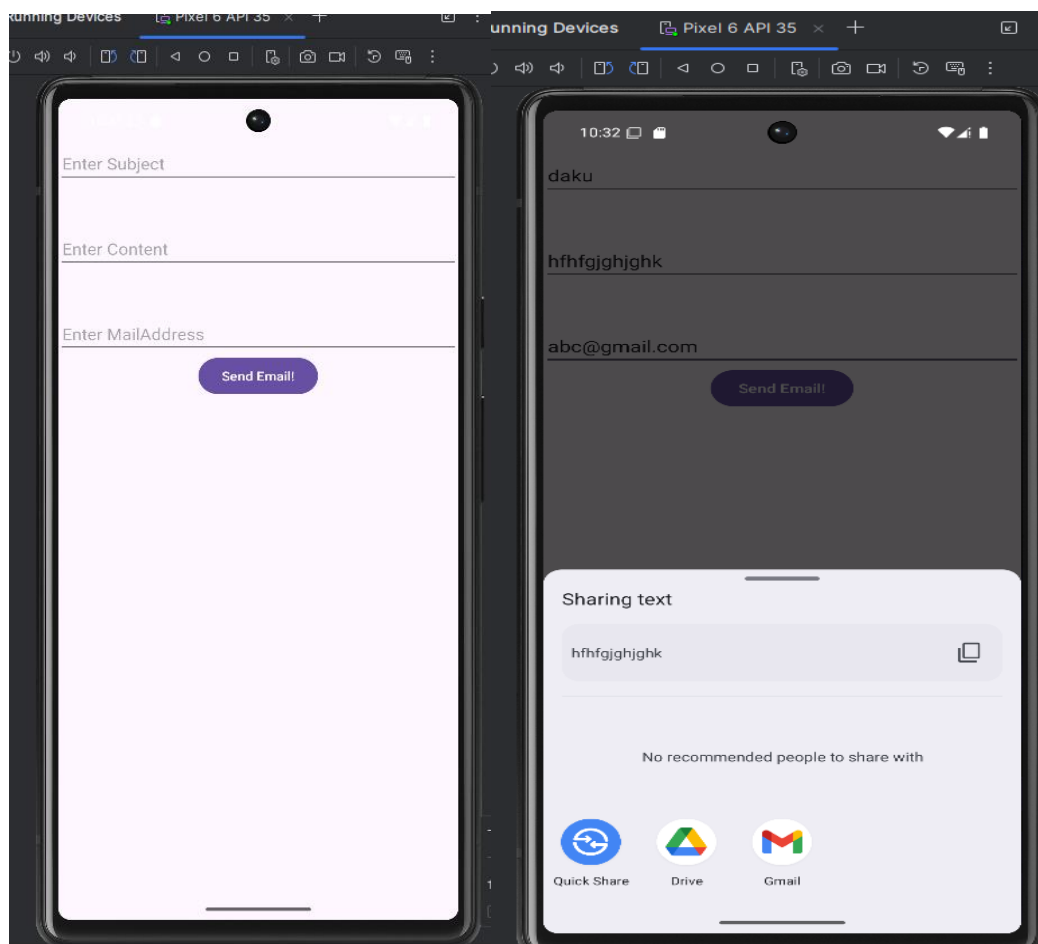
    });
}
    public void sendmail(String subject,String content,String
to_email){

        Intent intent=new Intent(Intent.ACTION_SEND);
        intent.putExtra(Intent.EXTRA_EMAIL,new String[]{to_email} );
        intent.putExtra(Intent.EXTRA_SUBJECT,subject);
        intent.putExtra(Intent.EXTRA_TEXT,content);
        intent.setType("message/rfc822");
        startActivity(Intent.createChooser(intent,"Choose email
content"));

    }

}

```



Program Number 12:

Display Map based on the Current/given location.

XML code

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    tools:context=".MainActivity">
    <TextView
        android:id="@+id/locationTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="18sp" />
</LinearLayout>
```

Java Code:Main_activity.java

```
package com.example.gps;

import android.content.pm.PackageManager;
import android.location.LocationListener;
import android.location.LocationManager;
import android.os.Bundle;
import android.location.Location;
import android.widget.TextView;
import android.Manifest;
import androidx.activity.EdgeToEdge;
```

```

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity implements LocationListener {
    private LocationManager locationManager;
    private TextView locationTextView;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        locationTextView = findViewById(R.id.locationTextView);
        if (ContextCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_FINE_LOCATION) !=
PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this, new
String[]{Manifest.permission.ACCESS_FINE_LOCATION}, 1);
        }
        // Get a reference to the location manager
        locationManager = (LocationManager) getSystemService(LOCATION_SERVICE);

        protected void onResume() {
            super.onResume();
            // Request location updates from the location manager
            if (ContextCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_FINE_LOCATION) ==
PackageManager.PERMISSION_GRANTED) {
                locationManager.requestLocationUpdates(LocationManager.GPS_PROVIDER, 0, 0,
this);
            }
        }
        // protected void onPause() {
        //     super.onPause();
        // }
        // Stop receiving location updates when the activity is paused
        // locationManager.removeUpdates(this);
        // }

        @Override
        public void onLocationChanged(@NonNull Location location) {
            locationTextView.setText("Latitude: " + location.getLatitude() + ", Longitude: " +
location.getLongitude());
        }

        @Override
        public void onStatusChanged(String provider, int status, Bundle extras) {
            LocationListener.super.onStatusChanged(provider, status, extras);
        }

        @Override

```

```

public void onProviderEnabled(@NonNull String provider) {
    LocationListener.super.onProviderEnabled(provider);
}
@Override
public void onProviderDisabled(@NonNull String provider) {
    LocationListener.super.onProviderDisabled(provider);
}
}

```

Android Manifest.xml code highlighted code to be added

```

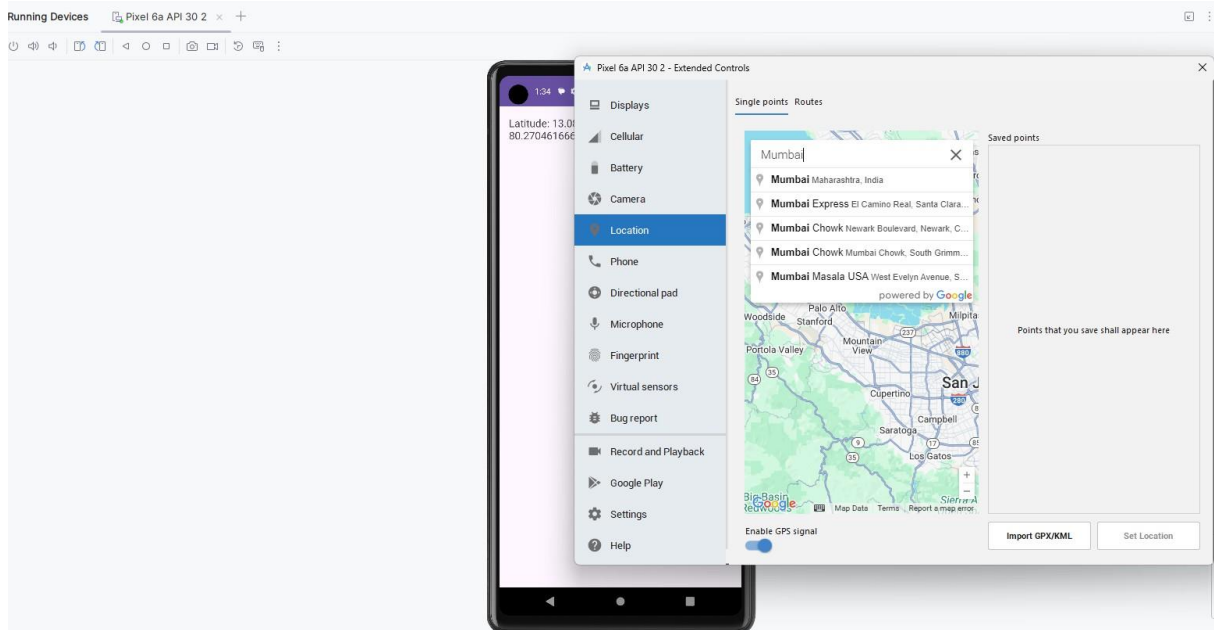
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.GPS"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>

```

While executing change the location to change the latitude and longitude. And click on set location and execute again.



Program 13

Create a simple application with login module(check user name, password) On successful login change rextview “Login Succesful”. On login fail alert using Toast “Login Fail”.

Java Code

```
package com.example.labprg3;

import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    private EditText inputUsername;
    private EditText inputPassword;
    private Button buttonLogin;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        inputUsername = findViewById(R.id.input_username);
        inputPassword = findViewById(R.id.input_password);
        buttonLogin = findViewById(R.id.button_login);

        buttonLogin.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                validateLogin();
            }
        });

        private void validateLogin() {
            String username = inputUsername.getText().toString().trim();
            String password = inputPassword.getText().toString().trim();

            if (TextUtils.isEmpty(username) || TextUtils.isEmpty(password)) {
                Toast.makeText(this, "Username and Password cannot be empty",
                    Toast.LENGTH_SHORT).show();
            } else {
                if (username.equals("admin") && password.equals("1234")) {
                    Toast.makeText(this, "Login Successful", Toast.LENGTH_SHORT).show();
                } else {
                    Toast.makeText(this, "Login Fail", Toast.LENGTH_SHORT).show();
                }
            }
        }
    }
}
```

XML code

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="20dp">

    <TextView
        android:id="@+id/label_username"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Username"
        android:textSize="18sp"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="50dp" />

    <EditText
        android:id="@+id/input_username"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:padding="10dp"
        android:hint="Enter your username"
        android:layout_below="@id/label_username" />

    <TextView
        android:id="@+id/label_password"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Password"
        android:textSize="18sp"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="20dp"
        android:layout_below="@id/input_username" />

    <EditText
        android:id="@+id/input_password"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your password"
        android:inputType="textPassword"
        android:layout_marginTop="10dp"
        android:padding="10dp"
        android:layout_below="@id/label_password" />

    <Button
        android:id="@+id/button_login"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Login"
        android:layout_marginTop="30dp"
        android:layout_below="@id/input_password"
        android:padding="10dp" />

</RelativeLayout>
```

Code explanation:

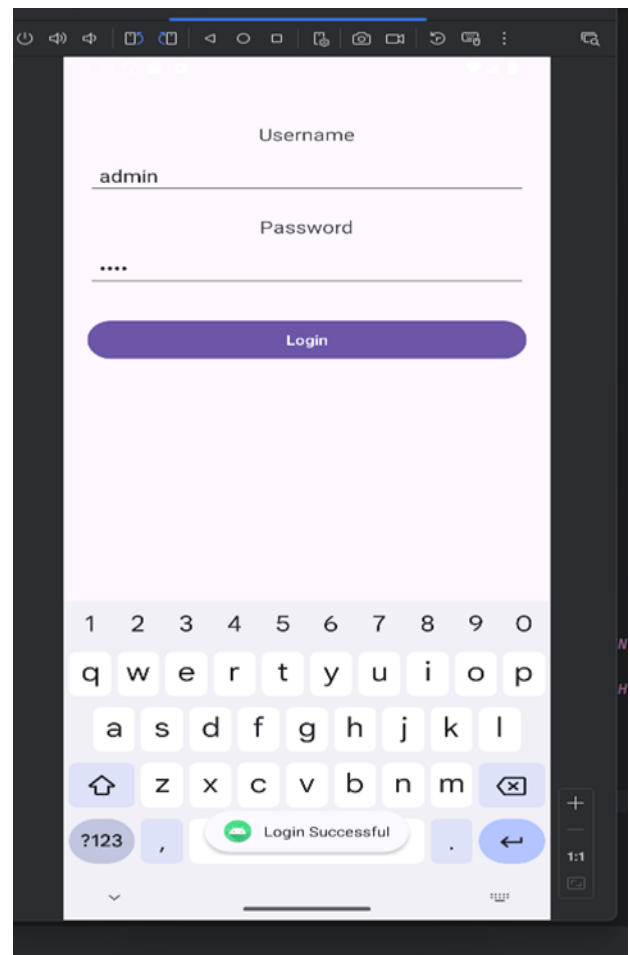
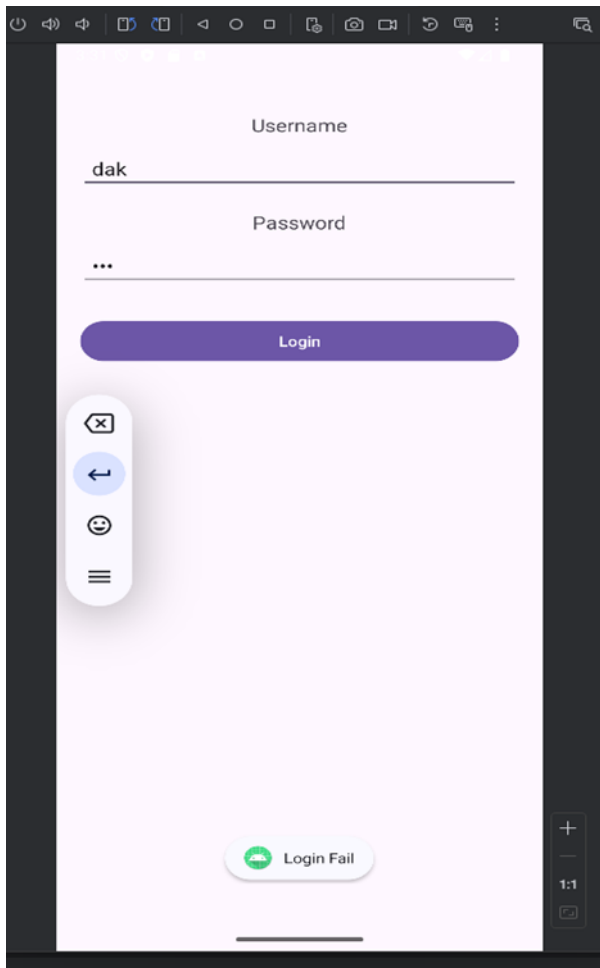
Java page: This Java code creates a simple login screen in an Android app. When the login button is clicked, it checks if the username and password fields are filled, and compares them to "admin" and "1234". If the credentials match, it shows a "Login Successful" message; otherwise, it shows "Login Fail".

XML Page: A Username label and input box

A Password label and input box

A Login button

All elements are aligned

**Program 14:**

Learn to deploy Android applications:

Steps to Deploy an Android Application

1. Prepare App (use Program 1 Hello world for this program) Optimize performance and test thoroughly. Ensure compatibility with various devices.

activity_main.xml Code:


```
<?xml version="1.0" encoding="utf8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">
```

```
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Hello World!"
app:layout_constraintBottom_toBottomOf="parent"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent"
android:textSize="30sp"/>
</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.java

```
package com.example.helloworld;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
public class MainActivity extends AppCompatActivity
{
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

2. Generate Signed APK (Android Package Kit):

In Android Studio, navigate to Build > Generate Signed Bundle/APK.

Follow the prompts to create a new keystore or use an existing one. A keystore is a binary file that contains a set of private keys.

Configure the build type (release) and signing configuration.

Generate the signed APK file.

3. Test Your Signed APK:

Before distributing your app, test the signed APK to ensure that the signing process didn't introduce any issues.

Install the APK on various devices and perform thorough testing.

Release on Google Play Console:

Sign in to the Google Play Console (<https://play.google.com/apps/publish>).

Create a new app entry if this is your first release or select an existing app.

Complete all the required information for the app listing, including the title, description, screenshots, and categorization.

Upload your signed APK file.

Set pricing and distribution options.

Optimize your store listing for search and conversion.

Once everything is set, click the "Publish" button to release your app to the Google Play Store.

5. Other Distribution Channels (Optional):

Besides Google Play, you can distribute your app through other channels such as Amazon Appstore, Samsung Galaxy Store, or third party app marketplaces.

Each distribution channel may have its own requirements and submission process, so be sure to follow their guidelines.

6. Monitor and Update:

Keep an eye on user feedback and app performance metrics through the Google Play Console.

Regularly update your app to fix bugs, add new features, and improve user experience based on feedback.

